

Python For Data Science Project Report

Martin Lau, Andreea Patarlageanu

1 Introduction

This report will outline the general steps for how we tackled the Kaggle data science challenge. The goal of the competition is to predict whether or not some users will churn in the window of 10 days, after "2018-11-20".

2 EDA & Data Aggregation

2.1 Initial Data Exploration

Our dataset contains 17499636 rows, and 19 columns, representing event-level user activity logs from a music streaming service. Each row represents a single user action/page visit. We have a record of user behavior from 2017-10-14 to 2018-11-20. The 19 columns are detailed in our *EDA* Jupyter Notebook. After an inspection of the unique values, we reordered the columns in a more intuitive way and started performing feature engineering. Since our dataset is organized in an event-level structure, we need to aggregate these events into user-level features, in order to be able to predict churn.

2.2 Train vs Test Set Differences

It is important we check if there are any large differences between our train and test sets. We observe that in the train set, there are "Cancellation Confirmation" and "Cancel" in the 'page' column. We infer that this should be how we determine whether a user has churned or not. In the test set, there are people signing up for the service, and not the other way around. Moreover, there are different values in the 'auth' column. The test set has "Logged Out" and "Guest", while all users in the train set are Logged In. An examination of the "Guest" rows revealed that they share the same userID.

2.3 Missing Values Analysis

We performed a missing values inspection, and we observed that there are 3208203 missing values in each of the 3 columns: 'song', 'artist', 'length'. As expected, all the missing values occur in the exact same rows for all 3 columns. Since the names of the songs, artists and the lengths shouldn't be correlated to churn, we decide to not remove these rows from our dataset, but fill the empty cells with 0s.

2.4 Target Variable Creation

We consider a user to "churn" if they complete the cancellation process during the observation period. This event appears in our dataset as 'Cancellation Confirmation' in the 'page' column. Thus, we create a new column, named 'churned': 0 if the user did not churn in the observation period and 1 otherwise. The resulted target variable shows significant class imbalance: 22.3% churned vs 77.7% not churned.

2.5 Data Aggregation

From the original form of our data, it was impossible to predict whether a user will churn from one user activity log alone. Hence, we aggregate all user activity logs by `userID`. To reduce the

dimensionality, we also ignore some redundant features, such as name, location, and artist/song names listened to in each session action, as these are considered to be independent to whether somebody will churn or not.

2.5.1 Basic and advanced aggregation features

We outline basic aggregations features:

1. **General information:** Gender (binary 0/1), Registration timestamp, Subscription level (first and current)
2. **Session information:** Number of sessions (unique session IDs), average session length, number of songs played, unique songs played, unique artists listened to, total and average song length
3. **Account information:** Days active (from first to last activity), membership length (from registration to observation end), days since last activity, subscription level changes (downgraded flag, days on current level)
4. **Page interaction counts:** Friends added, thumbs up/down counts, playlists created, help visits, error counts, settings visits, cancel page visits, logout count, home/about visits, advertisement roll count, downgrade/upgrade attempts, total page views

We gathered these inside our `aggregate_features_improved` function inside the `utils.py` file. We also performed some more advanced aggregation:

1. **Engagement metrics:** Songs per session, songs per day (overall and recent), artist diversity ratio, artist exploration rate, engagement rate (actions per song), actions per session
2. **Behavioral indicators:** Positive actions sum (thumbs up + playlists + friends), satisfaction ratio (thumbs up / total feedback), problem signals (help + errors + settings visits), ads per song ratio
3. **Activity trends:** Activity decline ratio (recent vs overall activity), session frequency trend (linear slope of daily sessions), session consistency (standard deviation of daily sessions), session length decline
4. **Recency-weighted features:** Weighted songs and sessions using exponential decay with 7-day half-life, prioritizing recent engagement
5. **Churn risk indicators:** Composite churn risk score, days since critical events (cancel page visit, help visit, downgrade attempt), recent problem signals (last 7 days)

We gathered these inside the `aggregate_features_improved2` function in the `utils.py` file. Since it includes more advanced aggregation methods, which cannot be solved with vector operations in numpy and pandas, it takes significantly longer to aggregate the test dataset.

2.5.2 Time Window Features

To capture temporal patterns in user behavior, we implement multiple time-window aggregations. This multi-window approach will allow the model to differentiate between short breaks in usage and actual disengagement:

- **7-day window (recent activity):** Songs played, sessions count, unique artists, average session length, cancel visits, help visits, thumbs down count. These features capture immediate user engagement levels and any (if existing) recent dissatisfaction signals.
- **14-30 day window (historical baseline):** Songs played, sessions count, unique artists. These capture a historical baseline in order to compare with the recent activity and detect declining engagement.
- **Comparative ratios:** We compute recent vs. historical ratios for songs, sessions, and artists to quantify engagement trends. If the values are below 1.0, they indicate declining activity, which is a strong churn predictor. The last session length was included to determine whether users were still highly engaged before churning.

2.6 Class balancing

The dataset is imbalanced based on churn. To solve this, we have two strategies. The first strategy is to set the parameter `class_weight='balanced'` for classes/models that allow (such as RandomForest, or DecisionTree). This solution is insufficient, and in confusion matrices, these models still tend to mislabel churned users as not churned. Thus, our second strategy is to apply the SMOTE library (the idea to use this was found [here](#)). This library synthetically generates data in the churned class to balance out the ratio between churned and non-churned users.

2.7 Feature Correlation with Churn

We wanted to inspect how each feature differs between churned and not churned users.

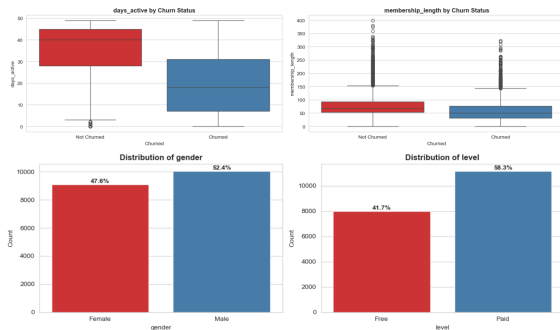


Figure 1: Distribution comparison of top discriminative features between churners and non-churners.

For ‘days_active’, the median is 40 days for non-churned users versus 18 days for churned users, showing that churned users become inactive earlier. More, ‘membership_length’ is shorter for churned users. Both groups have many outliers, suggesting that even a highly engaged user can churn. Gender shows almost identical churn rates (21.9% overall, 22.7% for males), so we exclude it from our predictions. For subscription level, paid users churn slightly more than free users (23.4% vs 20.8%). An explanation can be the higher expectations for paid memberships.

2.8 Undesirable features

After the aggregation, we have over 60 columns of feature data. However, not all features contribute positively to predictive performance, and can even harm model generalization. We identify and remove 3 categories:

- **Information Leakage:** Some features contain information that wouldn’t be available at prediction time. We removed features that used the observation end date or captured user behavior after they might have already decided to churn, like `ts_max` (last activity timestamp) and `days_since_last_activity`. These features would give good results during training, but would be unavailable or inaccurate for real-world predictions.
- **Multicollinearity:** Many engineered features have high correlation because of their derivation from common base features: (`songs_last_7_days`, `songs_per_day_recent`) and (`num_songs_played`, `total_length`) are nearly perfectly correlated. We remove features with pairwise Pearson correlation coefficient $|r| > 0.9$, and retain those with higher variance or interpretability. This reduces multicollinearity-induced instability in linear models and prevents redundant splits in tree-based models.
- **Statistical Insignificance:** We employ feature importance analysis using Random Forest’s Gini importance and permutation importance metrics to identify features with negligible predictive power. We remove features with importance scores below the 25th percentile threshold or near-zero variance. We apply L1 regularization (LASSO) in preliminary logistic regression models to automatically zero out coefficients of irrelevant features.

This filtering process reduces dimensionality from 69 to approximately 20-30 features, improves model interpretability, and can reduce overfitting risk, while maintaining or enhancing predictive performance.

3 Model Training & Evaluation

After we aggregated and balanced the event-level data into detailed user-level features, we proceeded to the model training. As mentioned, the goal is to identify which users will churn in the 10-day window following our observations, meaning after 20th November 2018. Having these in mind, we test multiple models to identify the most effective approach. We follow the reasoning:

- **Diversity of models:** We test linear models (Logistic Regression) and tree-based models (Random Forest, XGBoost, CatBoost) to capture different patterns in the data.
- **Evaluation metric:** We mainly use ROC-AUC as the evaluation metric, since it is robust to class imbalance and measures the model’s ability to distinguish between churners and non-churners across all decision thresholds.
- **Hyperparameter tuning:** We then attempt to tune our models to extract the best performance and accuracy from our data.

For each of the models, we trained on the adjusted feature set, and evaluated performance on the test set. The function `evaluate_model()` generated the predictions and created the submission file for the Kaggle competition with the `evaluate_model` function.

3.1 Training different models

Before we start training different models, we split our dataset into 80% train and 20% test. We established a baseline by first fitting a “vanilla” Logistic Regression, DecisionTree, RandomForest, XGB, CatBoost, leaving all the hyperparameters at their default values to our transformed dataset.

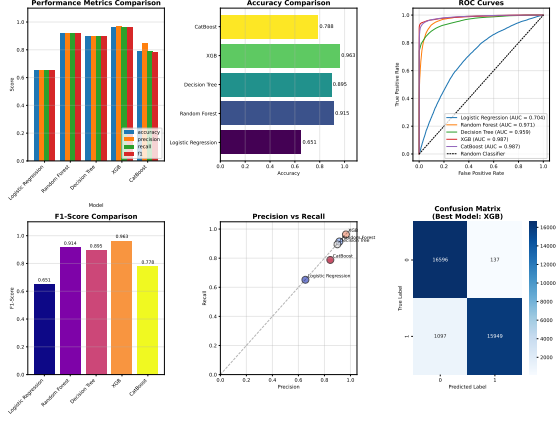


Figure 2: Comprehensive comparison of model performance across multiple metrics.

Figure 2 shows our different baseline evaluations on the dataset. Top left, is a global performance comparison for accuracy, precision, recall and f1 scores, which are further detailed in top middle, bottom middle and bottom left charts. The ROC curves are plotted in top right display our classifier’s ability to correctly identify positive cases versus incorrectly classifying negative cases as positive, across all possible classification thresholds.

The ensemble methods and gradient based boosting methods perform the best, obtaining nearly or over 90% accuracy and f1 score without any extra tuning. The confusion matrix also shows evidence of the high accuracy obtained through this method.

We would also like to highlight the effectiveness of the SMOTE class balancing.

	Predicted: Non-Churn	Predicted: Churn
Actual: Non-Churn	12,651	4,226
Actual: Churn	269	528

Table 1: Confusion Matrix for Baseline XGB, unbalanced classes

	Predicted: Non-Churn	Predicted: Churn
Actual: Non-Churn	16,596	137
Actual: Churn	1097	15949

Table 2: Confusion Matrix for Baseline XGB, balanced classes

The two tables compare the same vanilla model fitted to unbalanced (74.5% prediction accuracy) and balanced data (96.3% prediction accuracy). Namely, we see the apparent influence of class imbalance as many actually churned users have been mislabeled as not churned in the unbalanced case.

3.2 Hyperparameter Tuning with Optuna

For each classifier, we define an objective function that Optuna iteratively optimizes. The framework suggests hyperparameter combinations based on previous trial results, trains the model using 5-fold cross-validation, and evaluates performance using accuracy as the optimization metric. We configure Optuna

with `n_trials=100` per model, balancing optimization thoroughness with computational constrain. The TPE sampler intelligently concentrates search effort on promising hyperparameter regions by modeling the relationship between hyperparameter values and objective scores. We implement **MedianPruner** with `n_warmup_steps=5` to terminate poorly-performing trials early based on intermediate cross-validation scores, significantly reducing computation time without sacrificing optimization quality.

Below are the search spaces are tailored to each model’s characteristics:

Table 3: Hyperparameter Search Spaces for Each Model

Model	Hyperparameter Search Space
Logistic Regression	C: 10^{-4} to 10^2 (log scale) Penalty: L1, L2, ElasticNet (with conditional <code>l1_ratio</code>) Solver: saga Max iterations: 100–1000 Class weight: None, balanced
Decision Tree	Criterion: Gini, entropy Max depth: 3–30 Min samples split: 2–20 Min samples leaf: 1–10 Max features: sqrt, log2, None Class weight: None, balanced
Random Forest	Estimators: 100–500 Criterion: Gini, entropy Max depth: 5–50 Min samples split: 2–20 Min samples leaf: 1–10 Max features: sqrt, log2 Bootstrap: True (with max samples 0.5–1.0), False Class weight: None, balanced
XGBoost	Estimators: 100–500 Max depth: 3–15 Learning rate: 0.01–0.3 (log scale) Subsample: 0.5–1.0 Column sample by tree: 0.5–1.0 Min child weight: 1–10 Gamma: 10^{-8} to 1.0 (log scale) L1 reg (<code>reg_alpha</code>): 10^{-8} to 1.0 (log scale) L2 reg (<code>reg_lambda</code>): 10^{-8} to 1.0 (log scale) Booster: gbtrees, dart (with conditional sample type, normalization type, dropout 0.0–0.5)
CatBoost	Iterations: 100–500 Depth: 4–10 Learning rate: 0.01–0.3 (log scale) L2 leaf reg: 1–10 Border count: 32–255 Bagging temperature: 0–1 Random strength: 0–10

3.3 Model Comparison

After subjecting our computers to 2 entire days of hyperparameter optimization, we systematically compare all five models to identify the best performer for churn prediction.

For each model, we extract the best hyperparameters discovered by Optuna and instantiate final model objects. We ensure reproducibility by setting `random_state=42` across all models. For parallelizable models (Logistic Regression, Random Forest), we enable multi-core processing with `n_jobs=-1`. XGBoost is configured with `use_label_encoder=False` and `eval_metric='logloss'` to suppress deprecation warnings and specify the evaluation metric. CatBoost is set to `verbose=0` to minimize console output during training.

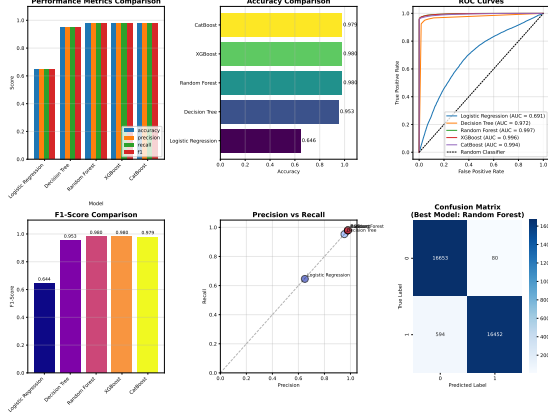


Figure 3: Comprehensive comparison of model performance across multiple metrics.

We evaluate all models on the held-out test set using the same testing functions used for the untuned models. For each model, we generate predictions and calculate accuracy, weighted precision, weighted recall, and weighted F1-score. The results are listed in Figure 3, in the same format as the untuned, vanilla models.

Based on the comprehensive evaluation, we select the model with the highest test set accuracy as our final model. The comparison reveals performance differences across metrics, with some models excelling in precision while others achieve higher recall. The selected model represents the best overall balance between all metrics for our churn prediction task.

4 Kaggle Testing

We take our tuned models and test them onto the Kaggle dataset. Firstly, a dummy submission with all 2904 userIDs labelled as '1' or '0' yields a public score of exactly 50%. This gives us a clue that within the public dataset (and likely the private one too, if they are split randomly), the class balance between churned and unchurned users is approximately equal.

We tune our `evaluate_model` function based on this observation: by including an extra parameter `p`, we can instead predict probabilities for the model, and define cutoff thresholds to have approximately submissions with balanced class predictions.

Despite the noticeable improvement of our models's test data compared to vanilla models in the figure above (accuracy extremely close to 99%), we find that our models perform rather poorly on the Kaggle challenge, reaching a submission performance of roughly 62-63%. We also test the vanilla models, which curiously perform better compared to the hyperparameter tuned ones, which is a definite sign of our overfitting of the models to the test data.

Curiously, we also observe that CatBoost has the best Kaggle

	Vanilla models	Tuned models
Log Reg	63.89%	60.8%
Dec Tree	60.0%	54.3%
RF	64.2%	63.57%
XGBoost	65.04%	64.5%
CatBoost	67.73%	65.8%

Table 4: Kaggle best submission performance comparisons, untuned vs tuned models

performance, so we continue to focus on that model to keep augmenting our score.

5 Conclusion

Through this project, we transformed 17.5 million event-level records into user-level features, and through feature engineering, we created more than 60 behavioral and temporal features, which captured user engagement patterns, activity trends and overall satisfaction.

Our methodology revealed how class imbalance can have a significant bad impact on the model performance. SMOTE improved XGBoost accuracy from 74.5% to 96.3% on test data. Moreover, removing correlated and leaky features improved generalization.

The big difference between the internal test performance, and the Kaggle score (the highest one achieved being CatBoost with 67.73%) shows clear signs of overfitting. Paradoxically, on Kaggle, the vanilla models overperform the hyperparameter-tuned ones.